Global Digital Wallet

PayPal-style wallet web application

Project Documentation: Requirements, Architecture, Tech Stack, Roadmap and Budget

Document version	1.0 (Draft for planning)
Date	28 September 2026
Platform	Web application (responsive)
Backend	Firebase (Auth, Firestore, Cloud Functions, Hosting)
Base currency	USD (multi-currency display and conversion)
Status	Planning: requirements confirmed, build not started

All cost figures in this document are planning estimates in USD. Items marked **(verified)** come from published vendor pricing checked in 2026; everything else is a range based on typical market rates and must be confirmed with vendor quotes before committing budget.

Contents

- 1. Executive summary
- 2. Confirmed requirements
- 3. Legal and compliance model (read first)
- 4. System architecture
- 5. Technology stack and tools
- 6. Data model
- 7. Money-movement rules and security
- 8. Feature specification (MVP)
- 9. Roadmap and team
- 10. Budget
- 11. Risks and open decisions
- 12. Next steps

1. Executive summary

The goal is a web-based digital wallet similar to PayPal: users sign up with Google, complete identity verification (KYC), hold a balance in USD, add money from cards and bank accounts, send and receive money peer to peer, request payments and issue invoices, and see a full transaction history. Other currencies are shown and converted at display time; the ledger itself is single-currency (USD).

The recommended build is a Next.js front end on Firebase, with **all money logic executed server-side in Cloud Functions** against a double-entry ledger in Firestore. Stripe handles card and bank funding, and a KYC provider handles identity verification. The biggest non-technical item is regulation: holding customer balances is a licensed activity in most jurisdictions, so production launch should use a licensed partner (Section 3).

2. Confirmed requirements

Area	Decision
Product type	PayPal-style wallet (not UPI; SIM binding is not applicable)
Platform	Web app
Backend	Firebase
Authentication	Google sign-in and sign-up, plus 2FA, plus KYC
Wallet model	Own in-app wallet balance
Currency	Global. One base currency (USD); others for display and conversion
Funding sources	Cards and bank transfers
MVP features	Wallet balance and add money; P2P send and receive; transaction history; payment requests and invoices
Purpose	Real production product (with licensed partner)

3. Legal and compliance model (read first)

Important: Storing user money in a wallet is generally regulated (money transmitter or e-money / prepaid payment instrument rules, for example RBI PPI rules in India, state money-transmitter licences in the US, e-money licences in the EU/UK). Building the software does not by itself give you the right to hold funds. This section is planning guidance, not legal advice; consult a payments lawyer in each target region.

Options to launch legally

Model	How it works	Pros	Cons
A. Licensed partner (recommended)	Funds are held by a licensed provider (BaaS / e-money issuer / Stripe Connect-type platform account structure). Your app is the UI and ledger layer.	Fastest, lowest legal burden, cheaper start	Revenue share or fees; dependent on partner; partner must approve your model
B. Own licence(s)	You obtain money-transmitter / EMI / PPI licences per region.	Full control and margin	Very expensive, slow (often 12+ months), capital and compliance staff requirements
C. Test mode only	Simulated money with Stripe test keys.	No licence needed; ideal for building and demos	Not usable by the public with real money

Compliance workstreams

- **KYC / AML:** identity verification at onboarding, sanctions and PEP screening, transaction monitoring, suspicious-activity reporting process.
- **Card data (PCI DSS):** never handle raw card numbers. Use Stripe Elements / Payment Element so scope stays at the lightest level (SAQ-A).
- **Data protection:** GDPR (EU/UK), CCPA (US), India DPDP Act. Privacy policy, consent, data-deletion flow, data-residency review for KYC documents.
- **Consumer terms:** Terms of Service, fee disclosure, dispute and refund policy, dormant-account policy.
- **Safeguarding:** customer funds held in segregated accounts (handled by the licensed partner in Model A).

4. System architecture

The browser never changes balances. It only calls authenticated Cloud Functions; those functions read and write Firestore inside transactions, and Stripe / KYC providers call back through verified webhooks.

Layer	Components	Responsibility
Client	Next.js + React web app (Firebase Hosting)	UI, Google login, 2FA prompts, calls to callable functions, Stripe Payment Element
Identity	Firebase Auth + Identity Platform (MFA)	Google sign-in, TOTP 2FA, session tokens (custom claims: kycStatus, role)
Application	Cloud Functions (2nd gen, Node.js + TypeScript)	transfer, createRequest, payRequest, createInvoice, startTopup, kycCallback, fxRefresh
Data	Cloud Firestore	users, wallets, ledger, transfers, requests, invoices, fxRates, auditLogs
Payments	Stripe (cards, bank debit / transfers, webhooks)	Funding the wallet, payouts / withdrawals, disputes
Compliance	KYC provider (Stripe Identity / Sumsub / Persona / Didit)	ID document and selfie verification, sanctions screening
Ops	Cloud Logging, Sentry, Secret Manager, App Check	Monitoring, alerts, secrets, abuse protection

Request flow: P2P transfer

- User taps Send, client calls `transfer(toUid, amountCents, idempotencyKey)` with the Firebase ID token.
- Function checks: authenticated, MFA satisfied, KYC verified, amount is a positive integer, limits not exceeded, recipient exists and is not the sender.
- Inside one Firestore transaction: re-check idempotency key, verify balance, debit sender, credit recipient, write transfer record and two ledger entries.
- Function returns status; both users get a notification; an audit log entry is stored.

5. Technology stack and tools

Category	Choice	Why / notes
Front end	Next.js, React, TypeScript, Tailwind CSS	Fast, SSR for public pages, strong typing for money code
UI / forms	shadcn/ui or Radix, React Hook Form, Zod	Accessible components, schema validation shared with functions
Hosting	Firebase Hosting (or Vercel)	CDN, HTTPS, preview channels
Auth	Firebase Authentication (Google provider)	Free up to 50,000 MAU for social sign-in (verified)
2FA	Identity Platform MFA, prefer TOTP (authenticator app)	MFA needs the Identity Platform upgrade (no licence fee). SMS OTP costs money per message; TOTP avoids that
Database	Cloud Firestore	Transactions, real-time updates, strict security rules
Backend	Cloud Functions for Firebase (2nd gen), Node.js 20+, TypeScript	All ledger writes; requires the Blaze (pay-as-you-go) plan
Card and bank funding	Stripe Payment Element, Stripe webhooks	Cards and bank debits without touching card data
Withdrawals / payouts	Stripe (where supported) or partner bank rails	Depends on the licensed-partner model
KYC	Stripe Identity, Sumsub, Persona or Didit	Compare on price, country coverage, and fit with your licensed partner
FX rates	ECB reference feed or Open Exchange Rates	Display and conversion preview only; ledger stays in USD
Email / push	Resend or SendGrid; Firebase Cloud Messaging	Receipts, invoices, security alerts
Invoices (PDF)	Server-side PDF generation (e.g. pdf-lib) stored in Cloud Storage	Downloadable invoices with signed URLs
Abuse protection	Firebase App Check, reCAPTCHA Enterprise, rate limiting	Blocks bots and scripted abuse of callable functions
Secrets	Google Secret Manager	Stripe and KYC keys never in code or client
Monitoring	Cloud Logging and Monitoring, Sentry, uptime checks	Error tracking, alerting on failed transfers and webhook errors
Testing	Firebase Emulator Suite, Vitest / Jest, Playwright	Rules tests, ledger invariants, end-to-end flows
CI/CD	GitHub, GitHub Actions	Lint, test, deploy to staging and production projects
Design / docs	Figma, Notion or Markdown docs, Postman	UI design, API contracts

6. Data model

All amounts are integers in **cents (USD minor units)**. Floating-point is never used for money.

Collection	Key fields	Client access
users/{uid}	email, displayName, kycStatus (none / pending / verified / rejected), mfaEnabled, displayCurrency, tier, limits, createdAt	Read own; write only displayCurrency
wallets/{uid}	balanceCents, currency = USD, updatedAt, version	Read own; no writes
ledger/{entryId}	txId, uid, type (topup, transfer_in, transfer_out, fee, refund, payout), amountCents, balanceAfterCents, createdAt	Read own; no writes
transfers/{txId}	fromUid, toUid, amountCents, status, note, idempotencyKey, createdAt	Read if participant; no writes

Collection	Key fields	Client access
paymentRequests/{id}	requesterUid, payerUid, amountCents, note, status (pending / paid / declined / expired), expiresAt	Read if participant; writes via functions
invoices/{id}	issuerUid, recipientEmail or Uid, lineItems[], totalCents, dueDate, status, pdfPath	Read if participant; writes via functions
topups/{id}	uid, stripePaymentIntentId, method (card / bank), amountCents, status	Read own; no writes
fxRates/latest	base = USD, rates{}, fetchedAt	Read all; written by scheduled function
auditLogs/{id}	actorUid, action, metadata, ip, createdAt	No client access (admin only)

Ledger invariants (enforced by tests)

- Sum of all user wallet balances plus the platform / fees account equals total funds received minus total paid out.
- Every transfer produces exactly one debit and one credit entry of equal amount.
- A wallet balance can never go negative.
- Replaying the same idempotency key never creates a second transfer.

7. Money-movement rules and security

Non-negotiable engineering rules

- No client-side balance writes. Firestore rules deny all writes to wallets, ledger, transfers and topups.
- Integer cents only; validate with Zod on both client and function.
- Every money-moving function is idempotent (client-supplied key, checked inside the transaction).
- Use Firestore transactions for multi-document changes; keep per-wallet write rate in mind (roughly one sustained write per second per document).
- Verify Stripe webhook signatures; credit the wallet only from a confirmed webhook event, never from the browser's success redirect.
- Hold new bank-debit top-ups as pending until the debit clears, to limit chargeback and return risk.

Account security

- Google sign-in plus mandatory 2FA (TOTP) before any transfer or withdrawal; step-up authentication for large amounts.
- KYC tiers: unverified users can log in but cannot move money; verified users get limits by tier.
- Velocity limits (per transaction, per day, per month), new-recipient cooling period, and anomaly flags.
- Security alerts by email for new device login, 2FA changes, large transfers.
- Encrypted at rest and in transit (Google-managed), least-privilege service accounts, Secret Manager for keys.
- Regular dependency scanning, annual penetration test, and a documented incident-response plan.

8. Feature specification (MVP)

Module	Features	Acceptance criteria (summary)
Onboarding	Google sign-up / login, profile, enable TOTP 2FA, start KYC, accept terms	User cannot transact until 2FA and KYC are complete
Wallet	USD balance, view in preferred display currency, add money by card or bank	Balance updates only after confirmed Stripe webhook; pending top-ups shown separately
Send money	Send by email or username, note, confirmation screen, receipt	Atomic, idempotent, instant; insufficient funds blocked
Receive money	Notifications, incoming transfer in history	Recipient sees credit immediately

Module	Features	Acceptance criteria (summary)
Payment requests	Request money from a user, pay, decline, expire	Only payer can pay; paid once; expiry job runs daily
Invoices	Create invoice with line items, send by email, pay from wallet, PDF download	Status flow: draft, sent, paid, overdue, cancelled
History	Filter by date, type, status; search; export CSV	Matches ledger exactly; pagination
FX display	Show balance and amounts in chosen currency, conversion preview	Uses latest cached rates with timestamp; clearly labelled as indicative
Settings and security	Change display currency, manage 2FA, sessions, close account	Security events logged

Post-MVP backlog: withdrawals to bank, refunds and disputes centre, QR payments, recurring payments, business accounts, multi-currency balances, admin back-office, mobile apps.

9. Roadmap and team

Phase	Weeks	Deliverables
0. Discovery and legal	1-3 (parallel)	Choose licensed-partner model, region list, KYC vendor, finalised scope, legal review kick-off
1. Foundation	1-3	Firebase projects (dev / staging / prod), CI/CD, Next.js skeleton, Google auth, MFA, Firestore rules and emulator tests
2. Wallet core	4-6	Wallet, double-entry ledger, transfer function, idempotency, transaction history
3. Funding	7-9	Stripe card and bank top-ups, webhooks, pending / cleared states, limits
4. Requests and invoices	10-11	Payment requests, invoices, PDF generation, email and push notifications
5. KYC and FX	12-13	KYC integration and tiered limits, FX rate refresh and display
6. Hardening	14-16	Security review, penetration test, load test, monitoring, docs, beta with real (small-limit) users

Estimated MVP duration: about 14-16 weeks with a small team, excluding time waiting for legal / partner approvals, which can run longer and should start on day one.

Suggested team

Role	Count	Focus
Full-stack engineer (lead)	1	Next.js front end, Firebase integration, overall architecture
Backend / payments engineer	1	Cloud Functions, ledger, Stripe, webhooks, tests
Product designer (part-time)	0.5	Flows, UI kit, accessibility
QA engineer (part-time)	0.5	Test plans, E2E automation, financial edge cases
Compliance / legal advisor (external)	As needed	Licensing model, KYC / AML policy, terms
Project manager (part-time)	0.25	Planning, vendor coordination

10. Budget

Figures are planning ranges in USD. Vendor prices marked (verified) were checked against published sources in 2026 and can change. Development rates vary widely by country and team; both a lean (India / offshore small team) and a Western-agency range are shown.

10.1 One-time build cost (MVP)

Item	Lean team estimate	Western agency estimate	Notes
Product design and UX	2,000 - 5,000	10,000 - 20,000	Flows, UI kit, prototypes
Front end development	4,000 - 9,000	20,000 - 40,000	Next.js app, all MVP screens
Backend and payments development	5,000 - 12,000	25,000 - 50,000	Functions, ledger, Stripe, KYC, FX
QA and test automation	1,500 - 4,000	8,000 - 15,000	Emulator tests, Playwright E2E
Project management	1,000 - 3,000	6,000 - 12,000	
Subtotal: build	**13,500 - 33,000**	**69,000 - 137,000**	
Legal set-up (payments lawyer, ToS, privacy, partner agreements)	3,000 - 10,000	8,000 - 25,000	Per region; grows with number of countries
Security review and penetration test	3,000 - 8,000	8,000 - 20,000	Before public launch
Brand, domain, misc. set-up	500 - 2,000	2,000 - 6,000	
One-time total (approx.)	**20,000 - 53,000**	**87,000 - 188,000**	Excludes licensing (Model B)

If you build the MVP yourself in test mode (Model C), the cash cost is mostly infrastructure and domain, roughly 0 - 500 USD, with your time as the main investment.

10.2 Monthly running cost by user scale

Component	~1,000 users	~10,000 users	~100,000 users	Basis
Firebase Auth (Google sign-in)	0	0	0 - 275	Free to 50,000 MAU, then from 0.0055 per MAU (verified)
2FA via TOTP	0	0	0	No per-use fee; SMS OTP would cost 0.01 - 0.46 per SMS (verified)
Firestore, Functions, Hosting, Storage	10 - 40	100 - 350	1,200 - 3,500	Estimate; depends on activity per user
Monitoring and error tracking (Sentry etc.)	0 - 30	30 - 100	100 - 400	Tier-based
Email / push notifications	0 - 20	20 - 100	150 - 600	Volume-based
FX rates API	0	0 - 30	30 - 100	ECB feed is free; commercial API optional
Support tooling and misc.	0 - 50	50 - 200	300 - 1,000	Helpdesk, status page, backups
Approx. monthly infrastructure	**10 - 140**	**200 - 780**	**1,800 - 5,900**	Excludes transaction fees below

10.3 Variable per-transaction and per-user fees

Fee	Typical cost	Who bears it / notes
Card top-up (Stripe)	Around 2.9% + 30 cents for US cards; higher for international cards and currency conversion	Confirm current rates and country-specific pricing on Stripe; usually passed to the user as a fee or absorbed in pricing
Bank debit top-up (ACH-type)	Roughly 0.8% capped per transaction in the US; varies by rail and country	Cheaper than cards but slower and with return risk
KYC verification	Stripe Identity: 1.50 per verification (as published by a competitor comparison); Didit: 0.30 each with 500 free per month (vendor claim); others quote-based	Per new user; one-time. Verify current price directly with the vendor
Licensed-partner / BaaS fees	Set-up fee plus monthly minimum plus per-account or per-transaction fee (quote-based)	Largest unknown; get quotes from at least three partners

Fee	Typical cost	Who bears it / notes
Payouts / withdrawals	Rail-dependent, often flat fee plus small percentage	Usually charged to the user

Note: Firebase Cloud Functions require the Blaze plan with a billing account, so a card on file is needed even while usage stays in free tiers.

10.4 First-year budget scenarios

Scenario	Build and set-up	12-month running cost	Total year 1 (approx.)
Test-mode prototype (self-built)	0 - 500	0 - 600	0 - 1,100
Lean production launch (licensed partner, ~10k users)	20,000 - 53,000	2,400 - 9,400 infrastructure plus partner and payment fees	25,000 - 65,000 plus fees
Agency-built production launch (~10k users)	87,000 - 188,000	2,400 - 9,400 infrastructure plus partner and payment fees	90,000 - 200,000 plus fees

Plan an additional 15 - 20% contingency for scope changes and compliance findings, and budget separately for ongoing compliance staff, customer support and fraud losses once real money flows.

11. Risks and open decisions

Risk / decision	Impact	Mitigation
Regulatory: holding customer funds without a licence	Critical: legal action, forced shutdown	Use a licensed partner (Model A); legal review before any real-money launch; confirm your model is permitted by Stripe / partner
Ledger bug or double-spend	Critical: financial loss	Idempotency, transactions, invariant tests, reconciliation job, small launch limits
Fraud, chargebacks and bank-debit returns	High	KYC tiers, pending periods, velocity limits, monitoring, reserve funds
Firestore hot-document write limits for busy wallets	Medium	Design ledger writes carefully; consider sharded counters or a relational ledger (e.g. Cloud SQL) if volume grows
Multi-country KYC and payment-method coverage differences	Medium	Launch in one or two regions first; check vendor coverage per country
Vendor lock-in (Firebase / Stripe)	Medium	Keep ledger logic in isolated modules with clear interfaces
FX rate accuracy or staleness	Low-Medium	Show timestamp and label as indicative; convert only via explicit, quoted flows
Open decision: target launch countries	Drives licensing, KYC vendor, Stripe availability	Choose first two markets before Phase 1
Open decision: revenue model	Drives fee design	Options: transfer fee, FX margin, invoice fee, subscription for business accounts

12. Next steps

- Confirm launch countries and shortlist 3 licensed-partner / BaaS providers; request written quotes and check that a Stripe-based wallet model is allowed.
- Engage a payments lawyer for a licensing opinion covering the first markets.
- Create Firebase projects (dev / staging / prod) and Stripe test account; start Phase 1 (auth, MFA, Firestore rules, emulator tests).
- Build Phase 2 (wallet, ledger, transfer) in test mode and run the ledger invariant tests before adding any real funding source.
- Finalise KYC vendor after comparing price, coverage and partner compatibility.

Sources and assumptions

- Firebase Authentication pricing: free social sign-in to 50,000 MAU, SMS verification requires the Blaze plan and is charged per SMS, MFA and blocking functions require the Identity Platform upgrade (published Firebase pricing summaries, 2026).
- KYC pricing: figures for Stripe Identity and Didit come from a Didit comparison page dated March 2026 and are vendor-published; confirm directly with each provider.
- Stripe card and bank-debit fee levels, infrastructure ranges, development rates and legal costs are planning estimates, not quotes.